
Lazysite - Complete Feature Reference

Everything lazysite has and does, and why - as of v0.4.17

Friday 26 June 2026

Contents

1	What lazysite is	1
1.1	The single architectural idea	1
1.2	Design constraints	2
2	Part I - The content model and processor	3
2.1	The request lifecycle	3
2.2	Authoring: front matter	3
2.3	Authoring: the Markdown pipeline and fenced constructs	4
2.4	Layouts and themes (decision D013)	5
2.5	Generated outputs: registries, scan, search	5
2.6	Remote content and dynamic data	6
2.7	Caching	6
2.8	Render-time security	6
3	Part II - Authentication and identity	8
3.1	The session model	8
3.2	Credentials	8
3.3	OAuth 2.1 - the AI web path	9
4	Part III - Authorization: capabilities, ACLs, and the deny-list	10
4.1	Capabilities	10
4.2	Per-file ACLs	11
4.3	The deny-list	11
4.4	Audit trail	11
5	Part IV - Payment (x402)	12
6	Part V - Publishing and management surfaces	13
6.1	The manager UI	13
6.2	The control API	14
6.3	File operations, locking, ACLs	14

6.4	Themes and layouts management	14
6.5	Plugins and form handlers	15
6.6	Backups and overlay install	15
6.7	Upload and download	15
7	Part VI - WebDAV publishing	16
8	Part VII - The MCP AI connector	17
9	Part VIII - Forms and delivery	19
10	Part IX - Installation, deployment, and operations	20
10.1	The installer	20
10.2	Hestia deployment	20
10.3	The dev server	20
10.4	Static site generation	21
11	Part X - Tooling, packaging, and supply chain	22
12	Part XI - The security model in one place	23
13	Part XII - Why it is built this way	24
14	Part XIII - Version history (feature timeline)	25
15	Part XIV - Roadmap	27

1 What lazysite is

Lazysite is a Markdown-driven website engine and lightweight CMS written in near-core Perl. You drop a `.md` file into a document root; on the first request the processor renders it to HTML through a layout/theme, caches the result as a sibling `.html`, and the web server serves that cache directly thereafter. There is **no build step, no database, and no application server** - just CGI scripts, a tree of Markdown, and a flat-file configuration.

Around that core sits a full publishing and management stack: a browser-based manager UI, a JSON control API, a WebDAV endpoint, an AI connector that speaks the Model Context Protocol (MCP), built-in cookie authentication with OAuth 2.1 for machine partners, per-file ownership and access control, an append-only audit trail, theming with self-service activation, forms with pluggable delivery, the x402 payment protocol, and a supply-chain-aware release pipeline.

1.1 The single architectural idea

One sentence explains the shape of almost everything below:

One enforced core, many thin transports.

Content is plain Markdown with YAML-ish front matter. The HTTP processor, WebDAV, the MCP connector, the control API (and the proposed Gopher/Gemini servers) are all **thin front-ends that translate a protocol into calls on the same shared action handlers**. Every rule - capability checks, per-file ACLs, the deny-list, path sanitisation, audit logging, cache invalidation - lives once, server-side, in that core. A new access surface adds *no authority of its own*; it inherits correctness for free. This is why the MCP connector's first version was ~300 lines, and why a lock taken over WebDAV blocks a save in the manager UI, an ACL set in the UI gates an MCP write, and every material change across all four doors lands in one audit log tagged with its origin.

A second through-line: **publishing by AI partners is a first-class use case**. Many features were driven by, and validated against, real Claude.ai and ChatGPT connector sessions building real sites.

1.2 Design constraints

- **Near-core Perl, minimal CPAN.** The processor runs on Text::MultiMarkdown + Template Toolkit + LWP + JSON::PP + Digest::SHA, all packaged by Debian. The only non-core *hard* extra is Archive::Zip (theme/zip handling); DB_File (rate limiting) and Template::Plugin::JSON::Escape (search index) are the other notable ones. The page processor (`lazysite-processor.pl`) is deliberately **self-contained** - it takes no project modules on its render path so it can be deployed as a single file.
 - **Run-in-place or packaged.** The same code runs straight from a git checkout via the dev server, or installs into `cgi-bin/` + docroot via a manifest-driven, upgrade-aware installer.
 - **Fail-closed on secrets, fail-open on availability.** Crypto/CSPRNG paths die rather than weaken; read-only settings consumers (`is-this-account-disabled`, rate stores) fail open so a corrupt file can't lock the operator out.
-

2 Part I - The content model and processor

The processor (`lazysite-processor.pl`) is the heart: a single CGI that maps a URL to a `.md/.url/cached.html` under the docroot, applies access control, renders, caches, and emits the response.

2.1 The request lifecycle

On each request the processor: localises `%ENV` (so per-request state can't leak under a persistent interpreter); resolves the URL; denies the `lazysite/` system directory outright; runs the **trust gate** (below); checks the manager-path gate; sanitises the URI against traversal; runs the auth check, then the payment check, then a preview-cookie check; serves from cache on the fast path; otherwise renders the source. Managers bypass the cache so the injected admin bar is never baked into anonymous HTML.

2.2 Authoring: front matter

Front matter is the block between a leading `---` and the next `---`, parsed by a deliberately minimal hand-rolled YAML-subset parser (zero non-core dependency, and every value passes through a Template-Toolkit-directive stripper for safety). One level of matching surrounding quotes is removed (`title: "Welcome"` → `Welcome`, YAML semantics). Recognised keys:

Key	Effect
<code>title, subtitle</code>	Page title/subtitle → <code><title></code> , <code><h1></code> , meta description, TT vars
<code>ttml</code>	Per-page cache lifetime in seconds; emits <code>Cache-Control: public, max-age=N</code>
<code>register</code>	List of registries this page joins (<code>sitemap.xml</code> , <code>llms.txt</code> , feeds, custom)

Key	Effect
tags	Page tags, surfaced in <code>scan: /registry</code> objects
date	YYYY-MM-DD publication date for feeds and <code>scan: sort</code> (mtime fallback)
tt_page_var	Page-scoped Template-Toolkit variables (literal / <code>url: / scan: / \${ENV}</code>)
layout, theme	Per-page layout/theme override (name or remote URL)
raw	Run the Markdown pipeline but emit no layout wrapper (default <code>text/plain</code>)
api	Body is pure TT, no Markdown, no layout - for clean JSON endpoints
content_type	Explicit Content-type header (with <code>raw/api</code>)
query_params	Allowlist of URL query params exposed as <code>[% query.x %]</code> ; bypasses cache
auth	<code>required / optional / none</code>
auth_groups	Required group membership (also doubles as payment bypass groups)
payment + payment_*	x402 payment gating and its parameters
search	Include in the search index (defaults to the site <code>search_default</code>)
form	Names and enables a form on the page (must match <code>forms/NAME.conf</code>)

A single-pass memoised “peek” reads the front matter **once per request** (keyed by `path:mtime`) - historically the same file was opened five times.

2.3 Authoring: the Markdown pipeline and fenced constructs

Custom fenced blocks are expanded to HTML *before* `Text::MultiMarkdown` runs, in a fixed order: `forms` → `:::divs` → `includes` → `code fences` → `oEmbed` → `MultiMarkdown` → `Template Toolkit + layout` → `link fix-`

ups. Inline `<script>` blocks are protected from the Markdown engine and restored afterward, and spurious `<p>` wrappers that MultiMarkdown puts around top-level block HTML (`<p><section>...`) are stripped.

- **::: classname boxes** → `<div class="classname">...</div>`. Class names are allow-list-validated; the box body is itself run through Markdown so headings and lists inside a box render properly. The default layout's CSS provides `widebox`, `textbox`, `marginbox`, `examplebox`.
- **::: include** pulls another local file or remote URL inline, with extension-aware handling (`.md` → recursively rendered, code files → syntax-fenced, others → escaped). A `tTL=N` modifier lets a remote include drive page cache lifetime, and TT-variable source paths are resolved in a second pass. No recursion (loop-safe).
- **```lang code fences** → escaped `<pre><code class="language-LANG">`.
- **::: oembed** embeds YouTube/Vimeo/PeerTube/Twitter/SoundCloud (with endpoint autodiscovery) and **bakes the result into the cached page** - no client-side API calls.
- **::: form** renders an accessible HTML form from a compact name | label | rules grammar. Field types: `text`, `email`, `tel`, `date`, `time`, `number` (with `min/max`), `url`, `password`, `textarea`, and `select:a,b,c` (multi-word options supported); plus `required/optional`, `max:N`, `pattern:"..."`, `placeholder:"..."`. Each form carries an HMAC time-token, a honeypot field, and an inline fetch-based submit handler that swaps to a success message - wired to a delivery handler (see Forms).

2.4 Layouts and themes (decision D013)

Layouts and themes are split. A layout (`lazysite/layouts/NAME/layout.tt`) is the structural HTML/Template-Toolkit skeleton; themes nest beneath it (`layouts/NAME/themes/THEME/`) and supply CSS custom properties generated from `theme.json` into a `:root { --theme-... }` block. A theme declares which layouts it is compatible with and is **ignored (with a warning) against an incompatible layout** - a broken theme is cosmetic, a broken layout breaks every page, so they are governed by separate capabilities. Layouts and themes can be local or fetched from a remote URL (cached, sandboxed, assets bundled). If no layout is installed at all, an **embedded fallback layout** renders a complete, self-styled page - the site always renders. The manager has its own dedicated, non-themeable layout.

2.5 Generated outputs: registries, scan, search

Pages opt into **registries** via `register::`; each registry is a Template-Toolkit template (`templates/registries/NAME.tt`) rendered to an output file - `sitemap.xml`, `llms.txt`, `feed.rss`, `feed.atom`, or any custom one you drop in. A recursive page scan collects registered

pages; regeneration is TTL-gated (4h) and only happens on a real render. The **scan: directive** turns `scan:/path/**/*.*md filter=... sort=...` into an array of page objects (url, title, date, tags, excerpt, searchable) usable in any template - the basis for blog indexes, card grids, and the search index.

2.6 Remote content and dynamic data

A `.url` file contains a single URL; the processor fetches it, renders the remote body through the full pipeline, and caches it - with a TTL so the refetch happens on the *next* request after expiry, never blocking the current visitor. Site variables in `lazysite.conf` can likewise be `url:` (fetched JSON usable via TT) or `scan:` (directory scans). Allow-listed CGI environment variables interpolate into config (`${SERVER_NAME}` etc.; the untrusted `HTTP_HOST` is deliberately excluded). Query parameters declared by a page are exposed as `[% query.x %]` and make that response uncacheable.

2.7 Caching

Rendered HTML is cached as a sibling `.html`, served when newer than its source (or within a page `ttl`). Writes are **atomic** (temp-then-rename), **refuse zero-byte output** (an empty cache file would permanently shadow regeneration), and are **realpath-guarded** against symlink escapes. A separate content-type cache preserves custom headers across cache hits, and Template Toolkit keeps an on-disk compiled-template cache. The whole cache base can be relocated off the docroot via `LAZYSITE_CACHE_DIR` (used by the dev server's browse mode so it writes nothing into a tree it is merely viewing). `LAZYSITE_NOCACHE=1` forces a one-off uncached render.

2.8 Render-time security

- **Path traversal:** URI sanitisation rejects null bytes, `..`, and dangerous characters; every read and write `realpath`-checks that the target stays inside the docroot.
- **No directory listing:** a directory resolves only if it contains `index.md`, else 404 - there is no `autoindex` anywhere in the processor.
- **System-dir & sidecar deny:** `/lazysite/*` → 403; `*.brief` authoring sidecars → 404 (and excluded from scans/registries).
- **Template-injection defence:** front-matter values and resolved variables are stripped of `[%/%]`; every Template instance runs with `EVAL_PERL => 0`.
- **SSRF defence:** every outbound fetch (remote pages, includes, oEmbed, `url:` vars, remote layouts) is screened against loopback/private/link-local/metadata addresses before any network I/O.

- **Header spoof defence (the trust gate):** the processor deletes client-supplied X-Remote-*/X-Payment-* headers unless a trusted source set them (see Auth).
 - **Baseline response headers:** X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, Referrer-Policy: strict-origin-when-cross-origin, and Vary: Cookie; protected pages are no-store, private and never cached.
-

3 Part II - Authentication and identity

Authentication is provided by a thin wrapper CGI (`lazysite-auth.pl`) that sits in front of the processor and manager: Apache routes everything through it (`FallbackResource`), it validates a signed cookie, sets the trusted `X-Remote-*` headers, then execs the real target. The processor itself contains no auth code - it only consumes the header contract.

3.1 The session model

- **Signed cookies, no server-side session store.** The cookie carries an HMAC-signed `username:timestamp:groups` payload - a stateless, tamper-evident session token, `HttpOnly; SameSite=Lax, Secure` under HTTPS, 24-hour expiry, compared constant-time.
- **Trusted headers + the trust gate.** Identity reaches the processor only as `X-Remote-User/-Groups/-Name/-Email` (names configurable). These are trusted *only* when a trusted source set them: the wrapper sets a one-shot trust signal after validating the cookie, and the processor's trust gate strips the headers otherwise. This two-signal model lets the built-in auth and an external auth proxy (Authentik/Authelia) share one header contract while staying spoof-proof; the edge web server is also expected to `RequestHeader unset` them (defence in depth).
- **CSRF.** Manager writes require an HMAC-over-(user, hour-bucket) token, accepted via header, JSON body, or query param, with a one-hour grace. The gate is keyed on **HTTP method** (every POST is a write) rather than an action allowlist, so a new write action can't be left unprotected. Static-token (API) clients are *exempt* (no cookie $\Rightarrow$ no ambient authority $\Rightarrow$ no CSRF vector), and combining cookie + token auth is refused so the exemption can't ride a browser session.

3.2 Credentials

- **Store.** Flat files: `auth/users` (`username:hash`), `auth/groups`, and `auth/user-settings.json` (capabilities, expiry, TOTP, provenance). Editable by hand, the manager Users page, or `tools/lazysite-users.pl`.
- **Password hashing.** Salted iterated SHA-256 (100 000 iterations), constant-time verify, with transparent auto-rehash of any legacy unsalted hash on next login.

- **Machine tokens.** High-entropy `lzs_` bearer tokens (256-bit) used as the WebDAV/API/MCP password; stored hashed (single iteration suffices for a random secret, and avoids a 100k-iteration KDF on every WebDAV request), shown once, with a 24-hour default expiry.
- **Single-use secrets.** Setup/reset **claim** links and partner **pairing keys** are single-use, short-lived, hashed, and redeemed under a lock (no replay/races). The credential *holder* sets their own secret - the operator never sees it.
- **MFA (TOTP).** Optional RFC 6238 second factor (self-contained, no CPAN) plus recovery codes, with a replay-aware verification window.
- **Forgot-password.** Emails a setup link (gated on the SMTP plugin), always returning a generic response - no account/email enumeration.
- **Sub-user delegation.** A partner holding `create_sub_users` can mint scoped sub-accounts; onward delegation requires holding the capability itself (no escalation).

Every gate runs *after* credential verification, so disabled/expired/MFA states never act as an oracle for valid usernames or passwords. All randomness is CSPRNG (`/dev/urandom`) and **fails closed**.

3.3 OAuth 2.1 - the AI web path

Claude.ai's **web** connectors are OAuth-only (no static-bearer field), so `lazysite-oauth.pl` + `Lazysite::Auth::OAuth` implement a minimal OAuth 2.1 authorization server: RFC 9728 + RFC 8414 discovery metadata, RFC 7591 dynamic client registration, mandatory PKCE (S256), and access/refresh tokens. The consent model reuses lazysite's one-time-code pattern: the operator mints a single-use **connect code** from the Users page, the human pastes it at the consent screen to prove they may act as that partner, and the issued token resolves to **the same partner grant** (capabilities + ACLs) as a static bearer would. No secret is ever typed into the third party. Tokens are stored hashed, short-lived, and garbage-collected; the MCP server accepts either an OAuth access token or a `partner:lzs_static` bearer and converges both on one enforcement path.

4 Part III - Authorization: capabilities, ACLs, and the deny-list

Authorization is two layers - coarse per-actor **capabilities** and fine per-object **ACLs** - both enforced in the shared core, plus a hard **deny-list**.

4.1 Capabilities

Per-user boolean grants in `user-settings.json`, gating actions across the manager API, WebDAV, and MCP. `whoami` reports the caller's effective grant.

Capability	Gates
<code>manage_content</code>	Content read/write across WebDAV + MCP (defaults to the <code>webdav</code> grant when unset; set off for a theme-only partner)
<code>manage_themes</code>	Theme activation and authoring under <code>lazysite/layouts/**</code>
<code>manage_layouts</code>	Layout activation and structure authoring
<code>manage_config</code>	<code>config-set</code> ; WebDAV editing of <code>nav.conf</code> and per-form dispatch configs
<code>webdav</code>	WebDAV access overall; also gates a partner managing ACLs on owned content
<code>ui</code>	Interactive browser login / manager / auth-protected pages (on by default)
<code>create_sub_users / delegate_sub_user_creation</code>	Sub-account creation and onward delegation

Each MCP tool and control-API action declares its required capability; a token client is confined to the control-API subset regardless of the cookie-manager surface, and unknown actions are refused.

4.2 Per-file ACLs

A central store (`lazysite/auth/acls.json`) optionally maps a docroot-relative path to { owner, read:[...], write:[...] }. The model: **no entry means allowed** (the account's namespace scope governs), the owner is always allowed, and list entries may be a username or `@group`. ACLs only ever *narrow* access. They bind identically across WebDAV, the manager, and MCP via one shared check. Crucially, the **two auth domains** differ: a cookie **operator** inside the manager bypasses ACLs (edits anything), but a **token/WebDAV/MCP partner is never an operator** and is bound by per-file ownership exactly like any external author - the linchpin that stops external partners escalating. `@group` entries match a cookie user's groups; a token client carries none, so `@group` never matches it (the safe default).

4.3 The deny-list

A hard, exact-path deny set is never readable or writable through the content tools: the HMAC secret, the user/group/settings files, and **any *.pl script**. A config-driven layer adds blocked directories and extensions. The WebDAV authoriser denies the whole `lazysite/` subtree **except** three gated carve-outs: `nav.conf` (with `manage_config`), per-form `lazysite/forms/<name>.conf` (with `manage_config`, but never `smtp.conf/handlers.conf` which hold credentials), and `theme/layout` authoring under `lazysite/layouts/**` (with the `theme/layout` capabilities). `lazysite.conf` itself is never WebDAV-writable. The blocklist applies on **reads too**, so script source can't be fetched. Failures return a machine-readable kind: `blocked`, `blocked-config`, `not-found`, `permission`, `binary`, `too-large`, `invalid-path`, or `exists`.

4.4 Audit trail

A single append-only writer (`Lazysite::Audit`) records **material events only** - state changes and security grants, never browsing - to `lazysite/logs/audit.log`, used by every state-changing entry point. Each line is `ts | user | action | target | ip | status | origin [ | detail ]`; `origin` distinguishes `ui` (cookie), `api` (token), `dav`, and `mcp`; `save` is recorded as `create` or `edit`; failures record the reason. This is deliberately **non-overlapping with the access log** - it answers *who changed what, to what, when, from where, and the outcome*. The manager audit viewer paginates (50/page), filters by user and by target (one file's history), links a page target to its editor, and shows the failure reason on failed events.

5 Part IV - Payment (x402)

A page marked `payment: required` is gated behind the **x402** HTTP payment protocol. Verification follows the same trusted-header pattern as `auth`: an upstream payment proxy sets `X-Payment-Verified: 1` (stripped by the same trust gate unless trusted). Unpaid requests get `402 Payment Required` with an `X-Payment-Response` JSON header describing the terms (amount converted to the asset's smallest unit, USDC on Base by default), rendering a custom `402.md` if present. Authenticated members of a page's `auth_groups` bypass payment entirely - membership substitutes for per-article payment, reusing the auth-group machinery. A working demo ships (`payment-demo.pl`).

6 Part V - Publishing and management surfaces

The manager UI, control API, WebDAV, and MCP connector are four front-ends over the same action handlers (`lib/Lazysite/Manager/*`), the same lock store, the same ACL store, and the same audit log.

6.1 The manager UI

A set of ordinary lazysite pages under a dedicated manager theme, calling the control API over `fetch`. Access requires authentication and membership in a `manager_groups` group. The pages:

- **Config** - schema-driven site settings (driven by the processor's own `--describe` descriptor), active layout/theme dropdowns, and a plugin registry (tick to enable/disable discovered plugins).
- **Files** - a browser over the docroot with per-row metadata, create/upload/delete, a type filter, bulk select with zip download and bulk delete, an inline **permissions editor** (owner select + per-principal r/w chips drawn from a user/group picker), lock glyphs, and `.brief` sidecar controls.
- **Editor** - front-matter form + raw-YAML toggle + body editor + live preview, with a **collaborative edit lock** (auto-renewing, 5-minute timeout), **stale-lock take-over** (which refuses to clear a live WebDAV lock), and **mtime conflict detection**.
- **Nav editor** - drag-and-drop reorder, indent/outdent nesting, link-vs-heading toggle; saving rebuilds the all-pages cache (nav is on every page).
- **Plugins** - per-plugin config forms (password fields never returned), action buttons, and the **form handlers + form targets** UI.
- **Themes** - installed-themes panel (activate/deactivate/rename/delete), **preview** any theme in your session via a signed cookie, **upload** a theme zip, and **install from GitHub Releases** of the configured layouts repo.
- **Users** - add/remove/rename, set/clear passwords, group membership, per-account **capability toggles**, **2FA**, **Generate credential** (a one-shot `lzs_token`), WebDAV scope, and the **AI partner onboarding** flows (connect code for the web OAuth flow; pairing-key brief for Claude Code / scripts), plus account disable/enable/reassign. Sub-user management is scoped to the actor's own subtree.
- **Cache** - list cached pages (with orphan badges), invalidate one or all.
- **Audit** - the paginated, filterable audit viewer.

- **Backups** - list/create/download tarball snapshots.

When the manager is enabled, the processor injects a compact **admin bar** on site pages for managers (Manage, Edit-this-page, Sign out, a no-password warning).

6.2 The control API

A single CGI (`lazysite-manager-api.pl`), action by `?action=`, JSON responses. Two mutually exclusive auth shapes (cookie/manager via the wrapper, or `Authorization: Basic user:lzs_token`); the method-keyed CSRF gate; per-token capability gating; and a per-token rate limit (token-bucket, burst 200 / refill 20s^{-1} , HTTP 429 + `Retry-After`). The verbs cover file CRUD + lock/preview/upload/ download/zip, ACL get/set/remove, cache list/invalidate, allow-listed config-set, the full theme/layout management set, artifact manifest/validate, users/principals (proxied to the users tool), plugins/handlers/form-targets, nav read/save, backups, SM071 preview grant/clear, whoami, version, audit, and rotate-auth-secret (the mass-logout lever - rewrites the install HMAC secret, invalidating every session at once).

6.3 File operations, locking, ACLs

The shared `Manager::Files` handlers underpin every surface: `list` (with size, type, lock, ACL, and sidecar flags), `read` (refuses binary), `save` (lock + mtime- conflict + ACL checks, cache + registry invalidation; a `nav.conf` save clears all HTML), `delete` (no recursive delete), `mkdir`, and `move` (which carries the `.brief` sidecar + generated `.html` and **re-keys the ACL**). A single lock store (`manager/locks/`) is shared with WebDAV - a manager save respects a live WebDAV lock and vice-versa - and theme/layout activation takes an artifact-level lock across `validate`→`snapshot`→`flip`.

6.4 Themes and layouts management

Activation is a careful, reversible operation: take a lock, **validate** the candidate (theme.json present + non-empty compatible `layouts[]`; layout.tt compiles), optional **optimistic-concurrency** check against a content-hash digest, **snapshot** the outgoing version (with retention pruning), flip the pointer, **invalidate only generated HTML**, and **mirror theme assets** to the public path. Layout activation additionally enforces a compatible (layout, theme) pair. Themes can be deleted/re-named/uploaded (zip, with zip-slip protection and strict `theme.json` validation), or installed from **GitHub Releases** of a configurable `layouts_repo` (with a lazy per-release content preview). The content-hash **manifest** doubles as the optimistic-concurrency token and a drift detector.

6.5 Plugins and form handlers

Plugins are discovered by probing scripts that answer `--describe` (a JSON descriptor of config schema, actions, and provided capabilities), enabled via the `plugins:` config block, configured through generated forms (password fields never returned on read), and invoked via action buttons.

Form handlers (`handlers.conf`) define named delivery targets - `smtp` (envelope here, connection in `smtp.conf`, delivered by `plugins/form-smtp.pl`), `file`, or `webhook` (JSON or Slack format) - and a form is wired to one or more handlers by its `<form>.conf`. The credentials and destinations live in operator-only config; an agent can *reference* a handler but never see or set a destination.

6.6 Backups and overlay install

The manager Backups page (and the installer) take `tar.gz` snapshots of served content (excluding the `infra` dir) tagged `preinstall` vs `manual`, with strict name validation on download. This is the safety net for the **non-destructive overlay install**: lazysite can be laid over a live HTML/SSI site without losing content - the processor serves existing `.html/.shtml` directly and only renders a `<page>.md` when present, so migration is page-by-page, and the one dangerous delete (a shadowing `index.html`) fires only when it was the regenerable cache of a pre-existing `index.md`.

6.7 Upload and download

Multipart upload (multiple files, size-capped and rate-limited *before* the body is read, with per-file deny checks and atomic writes), streamed download (deny-checked, `Content-Disposition: attachment`), and multi-file **zip download**. A content-type table and an editable-text extension set decide what the editor treats as text vs a binary download panel (`.htaccess` is intentionally binary).

7 Part VI - WebDAV publishing

A self-contained CGI (`lazysite-dav.pl`) at `/dav`, reached directly (not through the cookie wrapper - it does its own HTTP Basic auth), advertising **DAV class 1 + 2**. It is off by default (`webdav_enabled`), refuses Basic credentials over plaintext unless HTTPS/loopback/explicitly allowed, authenticates against the user DB with a per-IP failed-attempt limiter and brute-force delay, and enforces account state, the webdav mechanism flag, the deny-list (on reads too), `dav_scope`, and per-file ACLs (resolving the user's groups from the group file). It implements OPTIONS/PROPFIND (Depth 0/1, ETags, a `lzs:sha256` live property computed only when requested and only under layouts)/PROPPATCH/GET/HEAD/PUT (with If-Match/If-None-Match conditionals)/MKCOL/DELETE/COPY/MOVE, and class-2 **locking** (exclusive, Depth-0, refreshable, per-user flood-guarded) on the lock store shared with the manager. Throttled writes and locked resources always carry a `Retry-After` (the documented retry contract). Standard clients work: `curl`, `rcclone`, `davfs2`, GNOME/KDE, and - because class-2 LOCK shipped from the start - Windows Explorer and macOS Finder. There is no machine-account *type*; a bot is just a user with `webdav:on`, `ui:off` and a scope.

8 Part VII - The MCP AI connector

lazysite-mcp.pl exposes site maintenance as MCP tools an AI client can call. Transport is **Streamable HTTP / JSON-RPC 2.0** (POST = request + JSON response; GET → 405; protocol 2025-11-25); `initialize/tools/list` are open for discovery and `tools/call` requires auth. It accepts the dual bearer shapes (static partner: `lzs_` or OAuth access token), challenges unauthenticated calls with a 401 + WWW-Authenticate pointing at the OAuth metadata, and **disambiguates** “sign-in incomplete” vs “credential expired/revoked”. A token client is never an operator, so per-file ACLs bind it as over WebDAV; each tool declares a required capability; reads are not audited, writes are recorded as material events.

The **27 tools**:

Group	Tools
Identity	whoami (id, capabilities, active layout/theme, full tool manifest, auth method + expiry)
Read	list_files, read_file, read_page (parsed front matter + body), list_pages, page_status (will my edit reach visitors?), search_files, preview_page (server-side public render), validate_page, audit_site, get_permissions, list_form_handlers, read_nav
Write	write_file (validates on write), create_page, delete_page (removes .brief, reports dangling refs), rename_page (update_links), replace_text (no silent clobber), copy_file, move_file, delete_file, set_permissions, bind_form, set_nav

Group	Tools
Site ops	activate_theme, activate_layout, invalidate_cache

`validate_page` runs pre-publish checks including a **public-data warning** (Wi-Fi passwords, post-codes, phone numbers); `audit_site` finds broken links, orphans, missing titles, stale HTML, and duplicate blocks; `preview_page` renders fresh **as a public visitor** so verification stays in-channel. Each tool carries MCP `readOnlyHint/destructiveHint/openWorldHint` annotations so clients drive per-call approval (ChatGPT Plus/Pro = read-only; Business/Enterprise get writes with an approval card). The connector is **supervised, not autonomous**: bound by capabilities, ACLs, the deny-list, and the client's own approval. It is walled off by construction - form/SMTP configs, auth files, scripts, and the manager are denied with a machine-readable `kind`; user administration, secrets, and credential minting are **not exposed at all**. An agent can *wire* a form to a vetted handler (`bind_form`) but never set a destination or credential.

9 Part VIII - Forms and delivery

A `:::form` on a page (named by the `form:` front-matter key) renders to an accessible, CSRF-token-and-honeypot-protected HTML form that submits via `fetch` to a handler CGI and swaps to a success message. Delivery is configured by the operator: the form's `<form>.conf` references one or more named handlers in `handlers.conf`, each of type `smtp` (with shared connection config in `smtp.conf` - sendmail or authenticated SMTP with TLS and a `password_file`), `file` (stored submissions), or `webhook` (custom JSON or Slack-formatted). The forms docs cover the field grammar, the webhook JSON contract, and SMTP setup. Credentials and destinations are operator-only and deny-listed from every publishing surface.

10 Part IX - Installation, deployment, and operations

10.1 The installer

`install.sh` is a thin shim over `install.pl`, a ~960-line manifest-driven, upgrade-aware, core-Perl installer. It reads a `release-manifest.json` (built from a **classification** ruleset that decides where each file lands) and tracks installed state in `lazysite/.install-state.json` (a SHA map). On upgrade, **code files are always overwritten**, but **seed files are preserved if the operator edited them** (detected by SHA), files dropped from the new manifest are removed if untouched, and **a backup is taken before any change** (with configurable retention). `--dry-run` previews the plan with zero filesystem changes; `--restore` rolls back to a backup and invalidates the cache; runtime state (auth, logs, locks) is never touched. Imperative post-steps create cgi-bin symlinks for plugin endpoints, mirror the manager CSS, and seed fresh installs.

10.2 Hestia deployment

A two-layer model: a Hestia Apache **web template** owns the vhost (survives rebuilds), and `install.pl` owns the code/seed deploy. The vhost wires `DirectoryIndex` → `cached HTML`, `FallbackResource` → the auth wrapper (not the processor directly - that would break login), a rewrite that fronts the real cgi-bin scripts with the wrapper, the **RequestHeader unset X-Remote-*/X-Payment-* trust-strip**, the `/dav ScriptAlias`, a `Require all denied` on `/lazysite/`, a `.briefdeny`, and **Options -Indexes** (no directory listing). A rebuild hook fixes perms for the `no-suexec www-data` model; a one-command **deploy** script folds the whole runbook; and a **fleet updater** discovers every lazysite site on the host (by its state-file marker) and updates them all, optionally refreshing templates first. (A Docker target is a placeholder, not yet implemented.)

10.3 The dev server

`tools/lazysite-server.pl` is a single-threaded HTTP host for local development. It **defaults to no-cache** (edits show immediately), takes `--docroot/--port/--processor`, routes `auth/-manager/WebDAV` exactly as Apache does, forwards all headers, and serves static files. Its **--auto-**

index mode (SM091) turns *any* tree of Markdown into a browsable site **writing nothing**: it generates a directory index (folders + pages, labels from front-matter titles, README linked) for directories lacking an `index.md`, injects a breadcrumb nav into every page, relocates the processor cache to `/tmp`, and suppresses scaffolding seeding (also forced off by `--no-seed`, and never done in a non-lazysite tree). It cleans up on `Ctrl-C/kill`. Crucially, auto-index is **dev-only and off by default** - the production path never lists a directory (processor 404 + Apache - Indexes), pinned by a test.

10.4 Static site generation

`tools/build-static.sh <scheme://host> [out]` renders every page with the correct base URL into a static tree (sources stripped from the output), suitable for GitHub Pages, Netlify, or Cloudflare Pages.

11 Part X - Tooling, packaging, and supply chain

- **Release manifest** (`build-manifest.pl`) - deterministic classification of every shipped file with SHA + size + bucket; dies on unmatched files or path collisions; `--check` verifies a manifest against disk.
- **SBOM** (`manifest-to-sbom.pl`) - a CycloneDX 1.6 software bill of materials with a component per shipped file and per curated dependency (with Debian/RHEL/Alpine package names and licences). The `--strict drift gate` scans every script for use/require and **fails the release** if a dependency isn't declared - so a new dependency can't ship unaccounted-for. Everything is `Artistic-1.0-Perl`, overwhelmingly `Perl-core`.
- **Release pipeline** (`release.sh`) - never touches `main`: clones fresh, checks out a commit, runs the **full test suite**, builds the manifest, runs the strict SBOM gate, builds a reproducible tarball via `git archive`, records a SHA sidecar, and tags + pushes `vX.Y.Z`. Tags are the only stable identifiers; since the 0.4.x line, `main` is unstable and carries unreleased work with no per-release bump commit.
- **Versioning** (`bump-version.pl`) - promotes `NEXT_VERSION` into `VERSION` and advances the next, once per release.
- **User admin** (`lazysite-users.pl`) - the full credential/account/MFA/claim/pairing lifecycle as a CLI and a JSON `--api` (used by the manager Users page).
- **Offline bundle apply** (`lazysite-bundle-apply.pl`) - applies a network-less agent's single-JSON publishing bundle, deny-list-validated, dry-run by default.
- **Coverage** (`coverage.sh`) - measures the CGIs even though tests run them as subprocesses, enforcing a floor.
- **Benchmark** (`bench.pl`) - a host-relative gate on the hot paths (render, token verify, password verify), failing only on a gross regression.

The test suite is large ($\approx 1\,658$ tests across unit, integration, journey, and lint tiers) and is a release gate, alongside a `perlritic` gate, a secrets gate, and the SBOM gate.

12 Part XI - The security model in one place

- **Header trust model.** The central threat is auth/payment header spoofing; the defence is the two-signal trust gate plus edge stripping. Headers are the universal contract so built-in and proxy auth interoperate without trusting the client.
 - **Operator obligations (by design).** Strip client trust headers at the edge; set `manager_groups` (empty = any authenticated user is a manager); set a password for every non-localhost account (empty-password accounts work only from loopback); use HTTPS.
 - **Two auth domains.** Cookie operators bypass ACLs inside the manager; token/ WebDAV/MCP partners never do - they are bound by per-file ownership.
 - **Secrets are operator-only on every surface.** SMTP/handler configs, the HMAC secret, user/-group files, and all `.pl` are denied everywhere; forms can be *wired* but never *credentialed* by an agent.
 - **No-leak invariants.** Credential check precedes all state gates; generic responses on forgot/-claim/exchange; single-use + locked redemption; hashed-at-rest secrets shown once; constant-time compares throughout; atomic, zero-byte-refusing, symlink-guarded writes.
 - **Session revocation.** Rotating the install HMAC secret invalidates every cookie at once (the manager “log out all users” button).
 - **No directory listing in production**, ever (processor 404 + Options -Indexes), tested.
-

13 Part XII - Why it is built this way

The recurring design principles, drawn from the feature-request record:

- **One enforced core, many thin transports.** Every front-end translates; nothing re-implements policy. This is the reason a third or fourth surface is cheap and consistent, and it is the explicit justification for the modular refactor (SM079) that made the MCP connector a thin layer.
- **Control by function, not account type.** There is no “bot account” - a partner is a user with capabilities, a scope, and ACLs. Capabilities grew from *real* needs (e.g. `manage_content` appeared when a partner needed themes-but-not-content).
- **Drafts, live, and backups are the same object** distinguished by a pointer - so “roll back a theme” is just “activate a backup”, no new verbs.
- **Reference, don’t read, for secrets** - agents operate on credentials they must never see (`bind_form`).
- **Safe by default, no mode to remember** - e.g. the overlay install narrowed a dangerous delete instead of adding an `--overlay` flag that could be forgotten; `auto-index` is dev-only so production can never accidentally list files.
- **The AI client is a fuzzer for the whole stack** - live partner sessions drove the ergonomics roadmap and surfaced latent processor bugs (block-Markdown in boxes, multi-word `select:`, the UTF-8 double-encode) that ordinary use had routed around.
- **Permanent decisions, not just deferrals** - the specs record *why* alternatives were rejected (Digest auth, `mod_dav_fs`, admin-chosen passwords, dead-property stores), so a future revisit must overturn a reason rather than rediscover it.

14 Part XIII - Version history (feature timeline)

Newest first; releases are git tags.

- **0.4.17** (2026-06-26) - Dev-server `--auto-index`: browse any tree of Markdown with zero writes; production never lists a directory, now test-locked.
- **0.4.16** (2026-06-25) - UTF-8 corruption fully fixed (the second encoding layer); `read_nav/set_nav` complete the page API.
- **0.4.15** - UTF-8 fix in JSON responders; front-matter quote stripping; page-aware MCP verbs (`create_page/delete_page/rename_page`); `validate-on-write`; MCP tools reference doc.
- **0.4.14** - Multi-word `select`: options; stale-lock take-over; file size in Files.
- **0.4.13** - Block Markdown inside `:::` boxes; `whoami` auth lifetime; audit targets link to the editor.
- **0.4.12** - Connector tools `preview_page`, full tool manifest in `whoami`, `copy_file`, `get_permissions`, `list_form_handlers/bind_form` (SM088); clearer 401s + kind; `nav-save` clears all caches; audit-log usability.
- **0.4.11** - More form field types; safer connector tools (`replace_text`, `search_files`, `page_status`, `read_page`, `list_pages`, `validate_page` with the public-data warning, `audit_site`); generated-index refresh; audit pagination.
- **0.4.10** - Non-destructive overlay install; content backups + Backups page.
- **0.4.9** - Audit records material events only; `invalidate_cache`; reliability with slower assistants (512 KB read cap).
- **0.4.8** - Client-neutral connector (Claude.ai + ChatGPT; hints + output schemas); block-HTML `<p>` unwrap; `manage_content` capability.
- **0.4.7** - OAuth 2.1 authorization server for the connector.
- **0.4.6** - One-click Claude.ai setup; injection-resistant onboarding briefs.
- **0.4.5** - Users/Groups page layout fix.
- **0.4.4** - Audit WebDAV reads (quietable); MCP-vs-API onboarding docs.
- **0.4.3** - Files unified rights editor; @group ACLs over WebDAV; WebDAV + MCP writes in the audit trail (shared modules).
- **0.4.2** - Files-manager UI v2; richer audit (origin/target); `Hestia lib/` fix; **MCP server v1**.
- **0.4.1** - Files-manager overhaul; field-report fixes (theme-asset mirror, mixed form targets, audit target).

- **0.4.0** - Modular refactor (SM079) + security hardening + conformance milestone (perlcritic gate, SBOM gate, bench/secrets gates, bump-version.pl, five-audience docs, coverage instrumentation, fleet updater).
 - **SM070-SM074** (rolled into the 0.4 line) - WebDAV publishing + per-user ACLs; WebDAV theme/layout management with self-service activation; self-service credentials + claims + TOTP + account expiry; per-file .brief sidecars; per-file ownership + ACLs.
 - **0.3.0** (2026-04-23) - Release tooling split; SBOM/manifest generated per release; first upgrade-aware installer.
 - **0.2.0-0.2.19** (2026-04) - Hardening + manager maturation: structured logging, the Config and Files apps, method-keyed CSRF, mass-logout, login rate limiting, the D013 layouts/themes reshape.
 - **0.1.0** (2026-04-21) - Initial release: the Markdown→HTML processor with TT layouts/themes and scan/include/oembed; built-in + reverse-proxy auth; forms with an SMTP helper; the web manager; the x402 demo; the dev server; the test suite.
-

15 Part XIV - Roadmap

Actionable now:

- **SM085 - Git backend / changesets** - put the docroot under git, commit on every write with the partner as author, expose history/diff/restore. The biggest remaining lever and the substrate for transactional changesets.
- **SM084-restore** - the in-manager “restore this snapshot” action (list/create/ download already ship).
- **SM083 - Access-log stats plugin** - an awstats/webalizer-style dashboard from the access log; keeps browsing analytics separate from the material-events audit.
- **SM075 - Wildcard multi-tenant hosting** - many ephemeral sites under one wildcard vhost, auto-provisioned, with promote-to-permanent.

Candidates / research:

- **SM086 - Pandoc-construct renderers** - datatables, charts, boxes, definition lists, citations rendered for the web from the same source that makes a branded PDF.
- **SM089 - 3D-rendered layout** - a WebGL layout category, proving the rendering substrate itself is pluggable.
- **SM090 - Social syndication / POSSE** - ActivityPub + AT Proto, lazysite as the canonical store (publishing-format slice first).
- **SM092 - Gopher + Gemini services** - read-only public front-ends over the same content tree, the natural next “thin transports.”

This reference was synthesised from the lazysite source, the starter/docs/ documentation set, the docs/feature-requests/ record, and the CHANGELOG, at v0.4.17. For the authoritative detail of any feature, read the cited script or doc; for the “why”, read the corresponding SMxxx feature-request.